

Direct N-Body Gravitational Simulation

High Performance Computing

Savorgnan Enrico
SM 320 0120

September 15, 2026

The following document is the final report for the High-Performance Computing Exam, at the University of Trieste, Italy. The exercise chosen is Exercise 1, *Direct N-Body Gravitational Simulation*.

Contents

1	Introduction	2
2	Problem Description and Deliverables	2
2.1	Problem Description	2
2.2	Deliverables	3
2.3	Hardware and Software Stack	3
2.4	Run Configuration	4
2.5	Statistical Treatment, profiling and checks	4
3	Single-Core Engine	5
3.1	Experiments	5
3.2	Performances	11
4	OpenMP and MPI Strategies	13
4.1	OpenMP Kernels	13
4.2	OpenMP Performances	15
4.3	MPI Description	15
5	Containerization	16
5.1	Dockerfile	16
5.2	Apptainer	17
6	Scalability Analysis	17
6.1	OpenMP Scaling	18
6.2	MPI Scaling	19
6.3	Vectorization	22
6.4	Container Scaling	22
7	Conclusion	23

1 Introduction

This document is the report for the High-Performance Computing Exam, at the University of Trieste, Italy. The exercise chosen is Exercise 1, *Direct N-Body Gravitational Simulation*.

The document is organized as follows: in Section 1 we will discuss the problem, the requests of the exercise, the hardware and software used; in Sections 3 and 4 we will discuss the algorithms implemented for the serial and parallel code, respectively; we provide insights about containerization in Section 5 and about the scalability of the implementations in Section 6. Final considerations and remarks conclude the report.

All the code used for the simulations is available at:

<https://github.com/enricosavorgnan/nbody-gravitational-simulation>

2 Problem Description and Deliverables

In this section we described at first the problem and the general setup and requirements for the code; we will then present the hardware and software specifications and we finally discuss the statistical treatments for the measures taken.

2.1 Problem Description

The topic of the exercise is the *N-body problem*, well known in astrodynamics, and deals with the evolution of the gravitational force between N celestial objects in three-dimensional space.

In the simple setting considered the masses are considered known and equal among all the objectives; moreover, the potential is softened and the conditions are open-boundary.

The gravitational force is thus

$$\mathbf{F}_i = Gm_i \sum_{j \neq i} m_j \frac{\mathbf{r}_{ij}}{(|\mathbf{r}_{ij}|^2 + \epsilon^2)^{3/2}}$$

where ϵ is the softening length, a value used to keep the force finite for close encounters.

The time integration uses a second-order leapfrog (Drift-Kick-Drift) scheme:

1. Drift the particles positions by a time $\Delta t/2$
2. Compute the accelerations in those half-step positions
3. Kick the velocities by Δt
4. Drift once again the positions by $\Delta/2$

This algorithm works in $\mathcal{O}(N^2)$; it is not the best algorithm at all but is interesting for the problem because it exposes clear opportunities of parallelization. In the code implementation we fixed: $\Delta t = 10^{-4}s$, $\epsilon = 0.05$, $G = 1$, $\forall i, m_i = 1$. The number of masses considered varies among the different experiments, as well as the total number of steps done.

The exercise required to check the correctness of the algorithm by exploiting the *energy conservation law*: said $\mathcal{T}$ the total kinetic energy and $\mathcal{U}$ the total potential energy, the total energy $\mathcal{E} = \mathcal{T} + \mathcal{U}$ is conserved, and should stay within a relative tolerance of 10^{-3} .

The goal of the problem is to improve and parallelize the baseline code provided, exploiting at best the cluster resources available.

2.2 Deliverables

The following deliverables are required:

- Hardware identification
- Software stack
- Compiler flags
- Specific run code configurations
- Quantitative verification measures and statistical treatments
- Scaling plots and profiling
- Containerization via Docker images and Apptainer.

The parallel structure is required to use both shared and distributed memory approaches, i.e., both OpenMP and MPI.

2.3 Hardware and Software Stack

All the measures were taken on a single node **GENOA** of the Orfeo cluster by Area Science Park.

The command `lscpu` on the Genoa-004 node shown:

Property	Value
CPU Model	AMD EPYC 9374F (Genoa, Zen 4)
Sockets	2
Cores per socket	32
Threads per core	2
NUMA nodes	2
L1 cache	32 KiB per core (L1i, L1d), 2 MiB total
L2 cache	1 MiB per core, 64 MiB total
L3 Cache	32 MiB per CCX, 512 MiB total
Interconnect	Infiniband for RDMA, 200 Gbit/s
ISA	SSE/AVX/AVX2/AVX-512
Microarchitecture Level	x86-64-v4/-v3/-v2

Regarding the L3 cache, a *CCX* is a Core Complex, meaning that some cores (in this case, 4) share the same L3.

RDMA stands for Remote Direct Memory Access.

As the software stack we used the following libraries

Library	Version
Compiler	GCC 14.3.1
OpenMP	libgomp bundled with GCC 14.3.1
OpenMPI	5.0.5
Singularity	4.3.1
PAPI	7.1.0.0

2.4 Run Configuration

The following are the flags passed directly to the compiler:

```
-O3 -march=native -Wall -Wextra -Wpedantic -ffast-math  
-flto -DNBODY_USE_DOUBLE -D_POSIX_C_SOURCE=199309L
```

The flag `-ffast-math` has been introduced to allow the commutative property in math summations and thus helping the compiler to vectorize some operations. We have not noticed any loss of precision due to the introduction of this flag.

the flag `POSIX_C_SOURCE` is required for the wall-time clock measurements.

Depending on the experiment (see Sec.3 for more information), we introduced some other flags:

- `USE_PAPI`: allows the storage of profiling information using PAPI, including L1 and L2 cache misses, instruction per cycles and total cycles per run. Default to *True*.
- `<path/to/PAPI/library>`: required since Orfeo does not provide PAPI profiler by default.
- `USE_MPI`: defines a specific version of the DKD algorithm tailored on MPI. Default to *True* in experiments where MPI is introduced, absent elsewhere.
- `N_RSQRT_LOOP`: total number of Newton-Rahpson approximation steps for the *fast-inverse square root* implementation. Default to 1.
- `BLOCK_SIZE`: size of the blocks for some specific acceleration kernel implementations. Default to 128.

We also have some configuration flags to pass at run-time.

Beyond the flags present also in the baseline code we introduced:

- `kernel`: the short-form for the acceleration kernel to use. It defaults to the baseline implementation when present, otherwise to some other specific kernels.
- `profiler[0|1]`: whether to store profiling information (beyond PAPI, also time walls). Defaults to 1.
- `profiler-path`: the path where to store the profiler.

2.5 Statistical Treatment, profiling and checks

We used the Energy Conservation check as the main verification procedure.

The check usually runs only at the beginning and at the end of each experiment, but to check the correctness of the different algorithm we always have launched some trials checking the energy at each step; we avoided doing this when measuring performances because it is very expensive, running in $\mathcal{O}(N^2)$ by itself.

We repeated each experiment 5 times, and the plots show consider the trimmed mean (i.e., we excluded the best and worst results) $\pm$ the standard deviation, which is usually negligible.

During experiments we figured out that the most expensive operation in the code was (unsurprisingly) the computation of the accelerations during the DKD step.

For the two reasons above, we decided to always plot (except where specified) the trimmed mean time or speedup computed *per-step*, and not *per-run*, since we thought being a more reliable and clearer measures for the algorithm performances: indeed, in this way, we isolated the cost of the acceleration-kernel computation, without “polluting” the measures with other operations like I/O, read/write, Still, we have profiled all the steps of the code.

Regarding profiling, we implemented a granular system that measures the time spent in reading the masses’ binaries, the computation of the initial total energy, the computation for each step of the Leapfrog algorithm, and finally the time of writing the output in the output binary file. We have also measured the total number of cycles, instruction per cycle, L1 and L2 cache misses using the PAPI profiler.

3 Single-Core Engine

In this section we discuss the implementation of a relevant number of different accelerations-kernel (from now on, just *kernels*), responsible for the computation of the step 2 in the DKD presentation algorithm in Sec.2.

We decided to define different experiments, depending on their goal. Each experiment has its own source code, its own Makefile and its own specific kernels; however, each of them has been run on the same initial conditions generated by setting the seed to 42, and computing the simulation with 10^5 spheres for 10 time steps.

We run the following five experiments: *AoS-vs-SoA*, *FMA*, *Loop-Unroll*, *Rsqrt-Impact* and a more general *Kernel*; each one is briefly discussed in Sec.3.1; the different performances are then discussed more in depth in Sec. 3.2.

3.1 Experiments

In order to improve the given serial kernel (*baseline*), we considered the following aspects: the introduction of the data qualifier `restrict` for all the positions $\{x, y, z\}$ and for all the accelerations $\{a_x, a_y, a_z\}$, the removal of the branch in the innermost loop (`if j != i`), the adoption of the Newton’s Third Law to halve the computation, the use of accumulators, blocks operations, loop-unrolling to vectorize operations and finally the use of the fast-inverse square root. Finally, we compared a couple of kernels based on Struct-of-Arrays data versus the same algorithms applied to Arrays-of-Struct.

Branchless and `restrict` The adoption of the `restrict` qualifier, suggested in the baseline code itself, is required to tell the compiler that the value pointed by restricted pointer will never be accessed by other pointers. This specifics causes an undefined behaviour in case of *pointer aliasing*, and allows for a slight speedup in the computation.

The same effect is due to the removal of the `if` condition: the resulting branchless operations help the compiler vectorizing more effectively the code. The kernel having just those two modifications takes the name of “naïve”; its speedup with respect to the baseline is in practice almost null (cfr. Fig.8b), mainly because the compiler in this simple case is smart enough for allowing the vectorization by itself; however, the naïve version is still to prefer because it has L1 and L2 Cache misses lower, respectively, of around 10x and 5x times, and represent a very good starting point for the upcoming more sophisticated kernels (cfr. Fig.9).

Fast-Inverse Square Root Fast-inverse Square Root (Rsqrt) is an algorithm developed at first in the late '80s. It exploits a “magic” bit manipulation for providing an approximate solution to the operation $1/\sqrt{x}$. This result can be then refined by doing some steps of the Newton-Raphson algorithm.

The implementation of Rsqrt follows closely the code proposed in the [Wikipedia page](#).

In the **Rsqrt-Impact** experiment we considered up to 3 steps of the Newton-Raphson scheme.

The times per step and the speedup with respect to the naïve implementation are shown in Fig.1.

As one can see, not surprisingly, the more steps done the higher the cost of the operation.

Considering the energy relative drift, we obtained:

Kernel	Speedup	Maximum Relative Drift
Naive (1/sqrt)	$1.00x \pm 0.00$	4.35e-08
Rsqrt (1 iter)	$1.88x \pm 0.00$	4.74e-06
Rsqrt (2 iter)	$1.49x \pm 0.00$	5.23e-08
Rsqrt (3 iter)	$1.23x \pm 0.00$	4.35e-08
Rsqrt+Third (1 iter)	$2.47x \pm 0.07$	4.74e-06
Rsqrt+Third (2 iter)	$2.18x \pm 0.10$	5.23e-08
Rsqrt+Third (3 iter)	$1.81x \pm 0.06$	4.35e-08

The last three kernels show the combination of the Rsqrt with the Newton’s Third Law kernel, that we present in the next paragraph.

As you can see, the case of just one N-R step shows a drift which is $\sim 100x$ higher than the others; however, it is still well below the required drift of 10^{-3} , thus in all the kernels we present now on we used the Rsqrt implementation with just one N-R step.

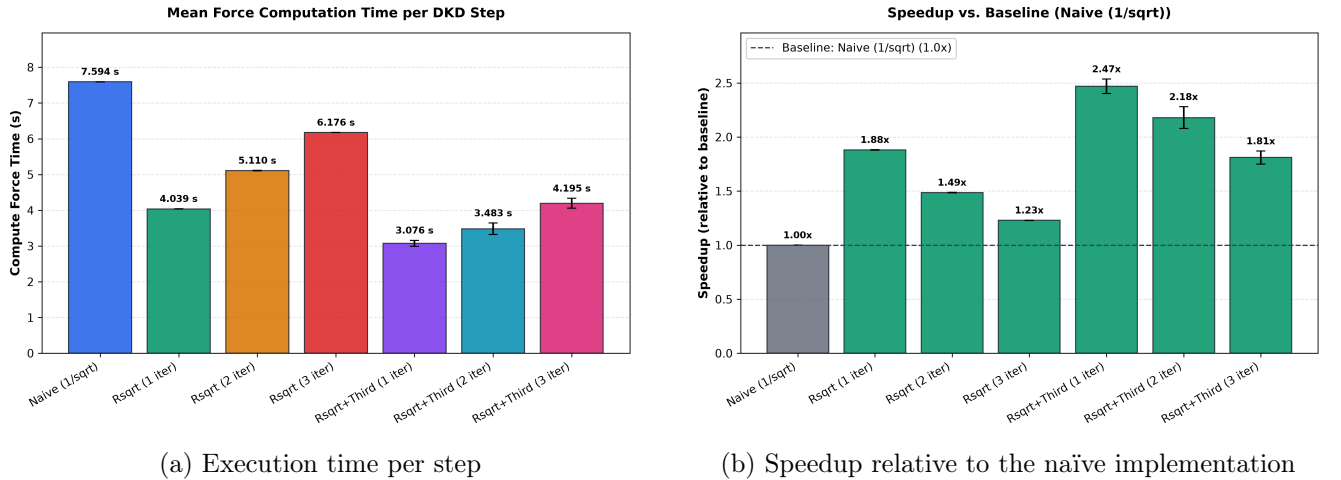


Figure 1: Performance comparison of the kernels in the Rsqrt-Impact experiment

Newton’s Third Law The Newton’s Third Law implies that $F_{ij} = F_{ji}$. Algorithmically speaking, this means that adopting this strategy can halve the computation by retrieving just one-way of the gravitational force (e.g., F_{ij}), and then just copy the value in the other way (i.e., F_{ji}).

As suggested in the exercise, this approach is fine in the serial setup, but becomes problematic when the inner loop is parallelized. We will discuss more in depth this aspect in Sec.4.

For now, it is just interesting to notice that, from Fig. 8a, the cost per step is not exactly the half of the

baseline. This is due to the very high number of both L1 and L2 cache misses as reported in Fig. 9. Those misses are mostly due to the fact that the “strange” behaviour of the kernel (with the innermost loop running or $j \geq i + 1$) does not allow the hardware prefetcher to pre-load the data (as in the naïve kernel); moreover, the amount of data to collect with $N = 10^5$ exceeds both the L1 and L2 caches, thus the CPU needs to flush the caches frequently. As a consequence, most of the time is spent on waiting for data, as shown by the $IPC \leq 1$.

Pairing this kernel with the blocks approach (cfr. next paragraph) allows a lighter work for reading/writing/flushing cache operations, unlocking the true benefits of the third law approach.

Blocks Doing operations by blocks is a classic approach in performances optimizations. The core idea is to re-use everything at most, i.e., to improve temporal and spatial locality by computing the accelerations for chunks of code.

The size of the blocks is set to 256 to perfectly fit (with some margin though) in L1.

The reason for the size is quite simple. Consider the case of working with double precision. During computation, the CPU needs to hold two blocks in cache simultaneously; for the first block, it needs 48 bytes per particle (the position $\{x, y, z\}$ and the acceleration $\{a_x, a_y, a_z\}$, each one occupying 8 bytes), while it needs just 24 bytes for storing the second block (since it only requires their position): the total footprint of each element of the block is 72 bytes.

The L1 cache size is 32KiB, thus the greatest power of 2 that is less than $32 \text{ KiB} / 72 \text{ B} \approx 440$ is exactly 256.

As one can see from Fig. 2, the blocks kernel by itself does not improve the computation time with respect of the naïve kernel, mainly because the prefetcher is good enough to vectorize perfectly the baseline code; instead, combining Blocks with Rsqrt and Third Law improves the performances, and indeed that kernel appears to be the best one, with a total speedup of up to 2.74x, while having a lower number of cache misses, and a IPC rate that doubles the baseline (cfr. Figs.3, 9). Blocks are thus a powerful approach; however they introduce some difficulty of implementation for parallel kernels.

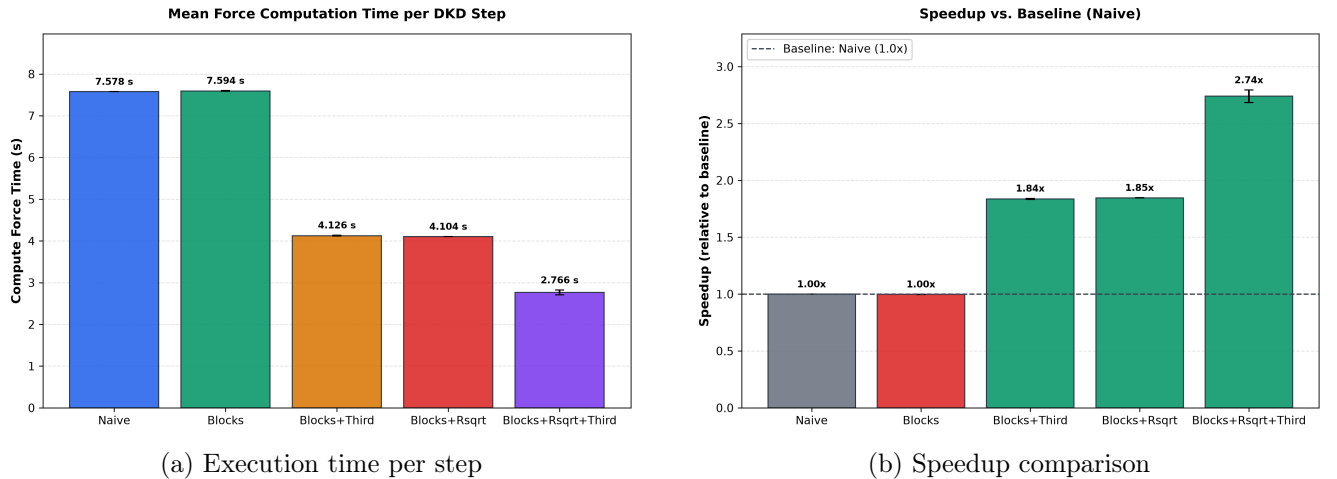


Figure 2: Performance comparison of the block-based kernels

Loop-Unroll The baseline kernel is, at its core, nested-for loops. We tried unrolling the inner loop using the GCC `#pragma GCC unroll N`, and tried with $N = \{2, 4, 8, \text{auto}\}$.

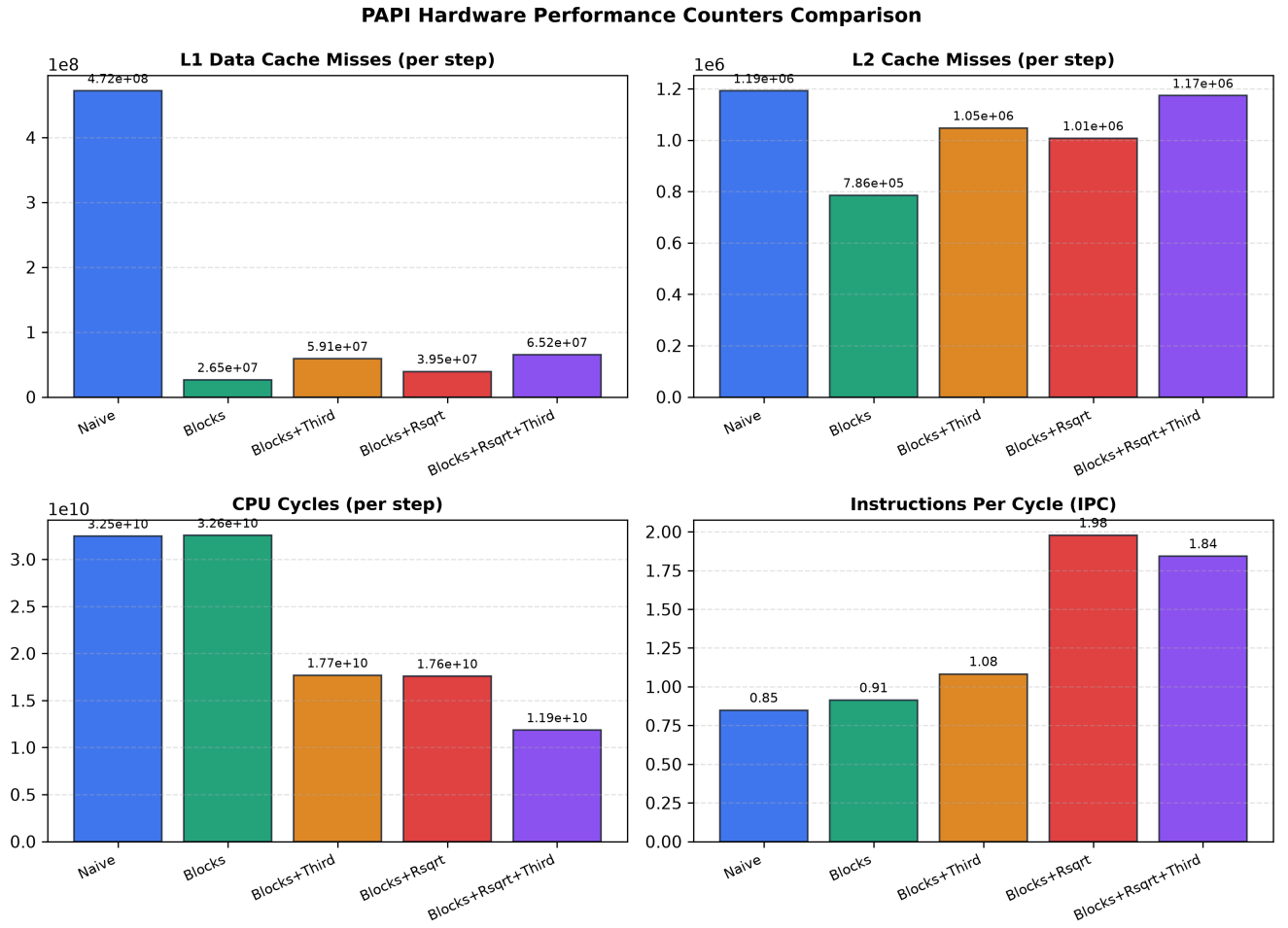


Figure 3: PAPI profiling results for the block-based kernels

In practice, we have not seen any relevant improvement with respect to the baseline (Fig.4), so we have not investigated more this path; the main reason for those results is that the compiler is smart enough to vectorize by itself the computation, limiting at all the benefits of such technique.

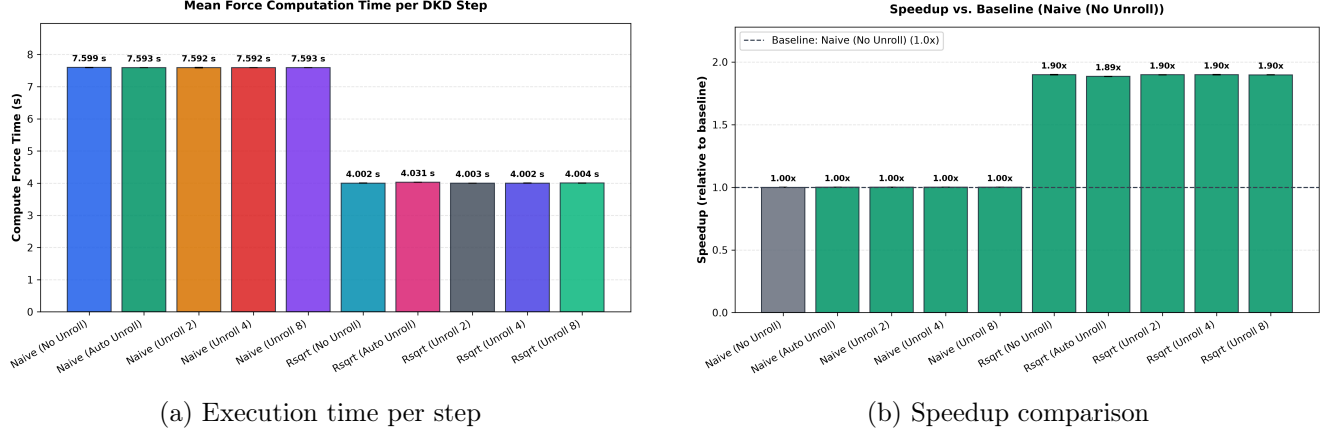


Figure 4: Performance comparison of the kernels in the Loop-Unroll experiment

Fused Multiply-Add FMA is a hardware instruction available in AVX-512 (and others) instruction sets that allows for doing the operation $A \times B + C$ in a single clock step, speeding up the computation. In practice, it consists of multiple accumulator units, mutually independent, that break the dependency chain due to the loop.

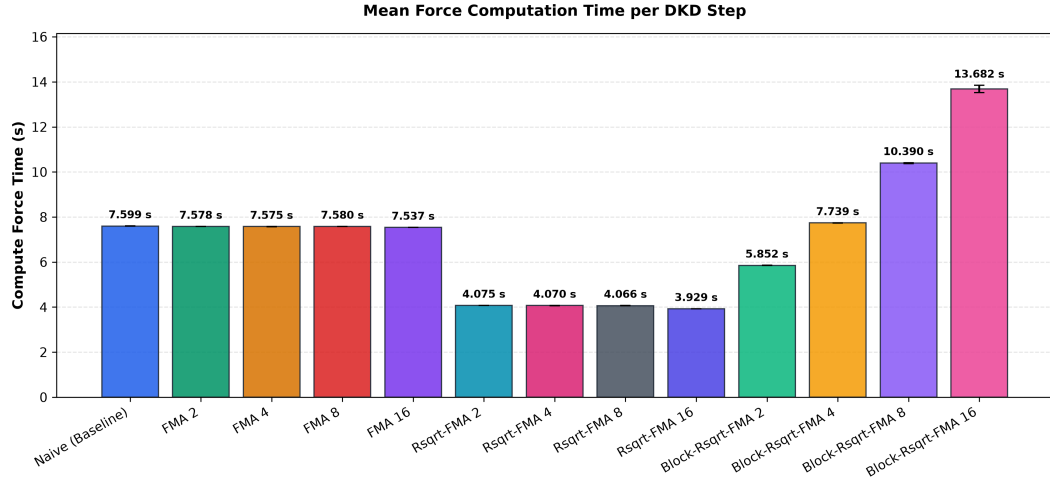
An FMA units can accept a new instruction at every single clock (thus having high throughput), the actual calculation takes more clocks to complete, showing an instruction-induced latency. By unrolling the inner loop and maintaining the multiple accumulators (in the code, `axi_0`, `axi_1`, `axi_2`, ...), the CPU can pipeline consecutive FMA instructions simultaneously without waiting, hiding the hardware latency. Once the inner loop terminates, the independent accumulators are simply summed together to get the final result.

The final version for the FMA experiment has partially been rewritten by using Claude Opus 4.6, substituting old versions that were running extremely slowly; some “memories” of the first FMA implementation can be found in the untouched kernels Blocks+Rsqrt+FMA (that indeed still perform poorly). The new version of FMA achieved nice good results when combined with Rsqrt, but not improving the Rsqrt kernel *without* FMA. As one can see, in particular from the non-optimized kernels with Blocks+Rsqrt+FMA, performances degradate at the increase of the number if accumulators used, becoming even worse than the naïve kernel when we included 8 accumulators or more (Fig.5).

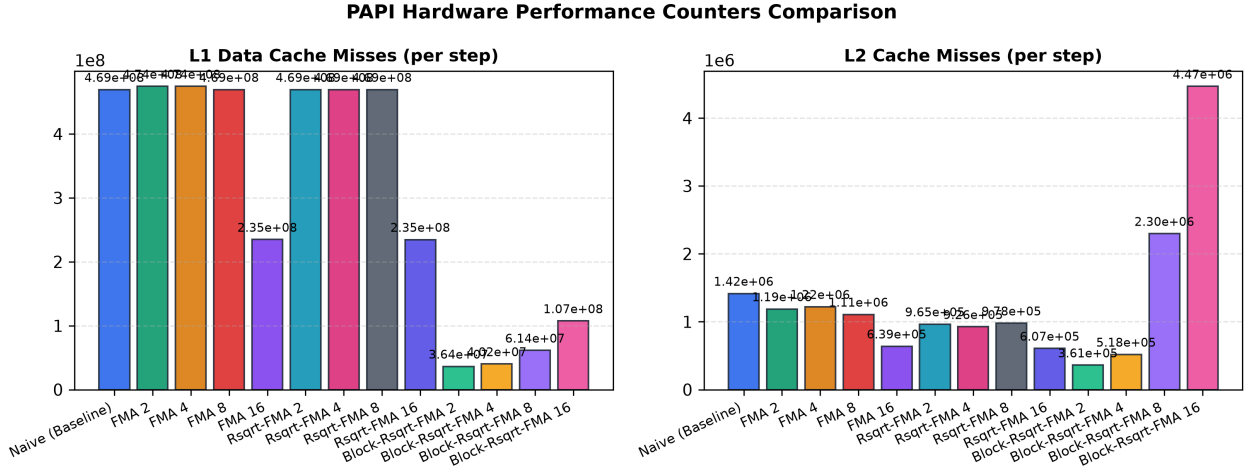
Moreover, the development of FMA, which is a CPU-level optimization, becomes critical when parallelizing the code; for this reason we thought it was a better idea to take note of those results but follow other paths for the parallel kernels.

AoS VS SoA The final aspect to discuss is the choice between an Array-of-Structs (AoS) or Struct-of-Arrays (SoA) approaches.

The baseline code provided exploited the latter. To test the performances of the two approaches we decided to re-write entirely the algorithm in order to work natively on AoS, and we compared it against SoA on the naïve and Rsqrt kernels.



(a) Execution time per step



(b) PAPI profiling results for cache misses

Figure 5: Execution time and cache profiling results for the FMA experiment

To be honest, both the kernels are built for working well on SoA kernels, so the comparison is unfair at its core: the algorithms applied to AoS are not vectorizable.

The PAPI report shows what we were expecting, i.e., high number of L1 cache misses and a very high number of CPU cycles per step, meaning that the CPU is continuously working for finding and loading data: the throughput is high, but its usefulness is pretty low (Fig.7).

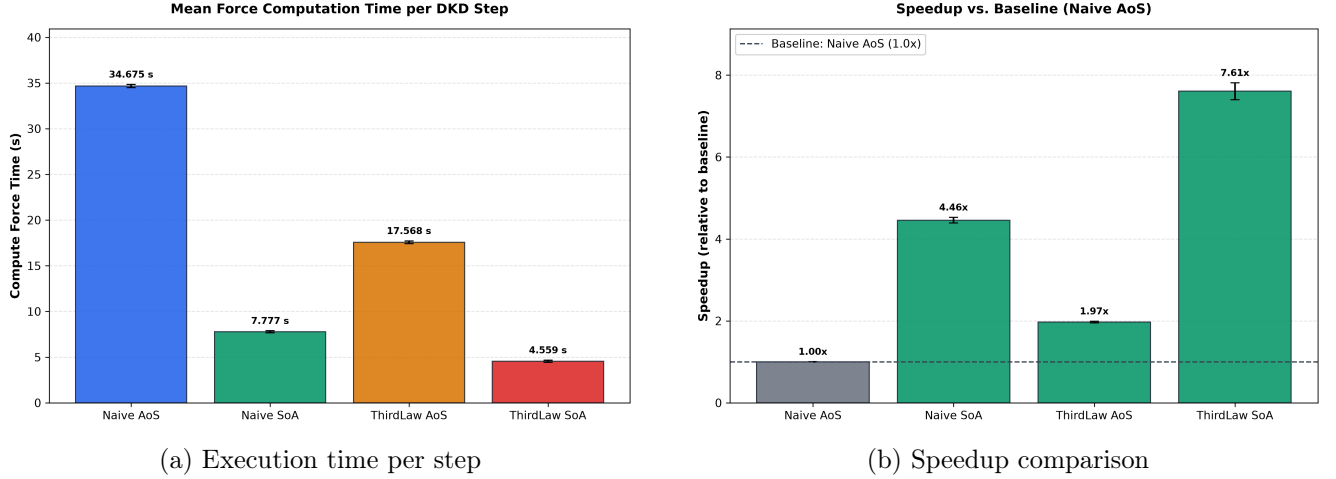


Figure 6: Performance comparison of AoS vs SoA kernels

3.2 Performances

We conclude the discussion presenting more details on the kernels we figured out to be more interesting for an efficient implementation in a parallel scenario.

As already anticipated in the previous paragraphs, we decided to leave the kernels using FMA, whose development has been particularly complicated, as well as the loop-unrolling kernels, since they have not shown any clear benefit; of course, we have also rejected working on AoS-based kernels.

The remaining approaches (the naïve, Third Law, Rsqrt, Blocks) were combined together and studied in our last experiment for the serial part, called “kernel” in the code.

Beyond what already previously said, it is worth noticing that all the kernels that do not use the fast-inverse square root show a maximum energy drift on the order of 10^{-8} , versus the 10^{-6} for the kernels using it. Standard deviations σ were almost null for kernels that do not use the Third Law, while the ones using it show $\sigma \propto 10^{-1}$ s.

We thought that the reason for this increase relates with spatial locality of the data, that in this case becomes relevant, being those types of kernels less compute-bounded and more communication-bounded. Helping Third Law with tiling and cache blocking reduces these results up to the 75%, reducing $\sigma \propto 10^{-2}$ s.

We provide the execution times and speedups for these kernels in Fig. 8 and the PAPI profiling results in Fig. 9, and below some information about speedups, with standard deviation and maximum relative energy drift.

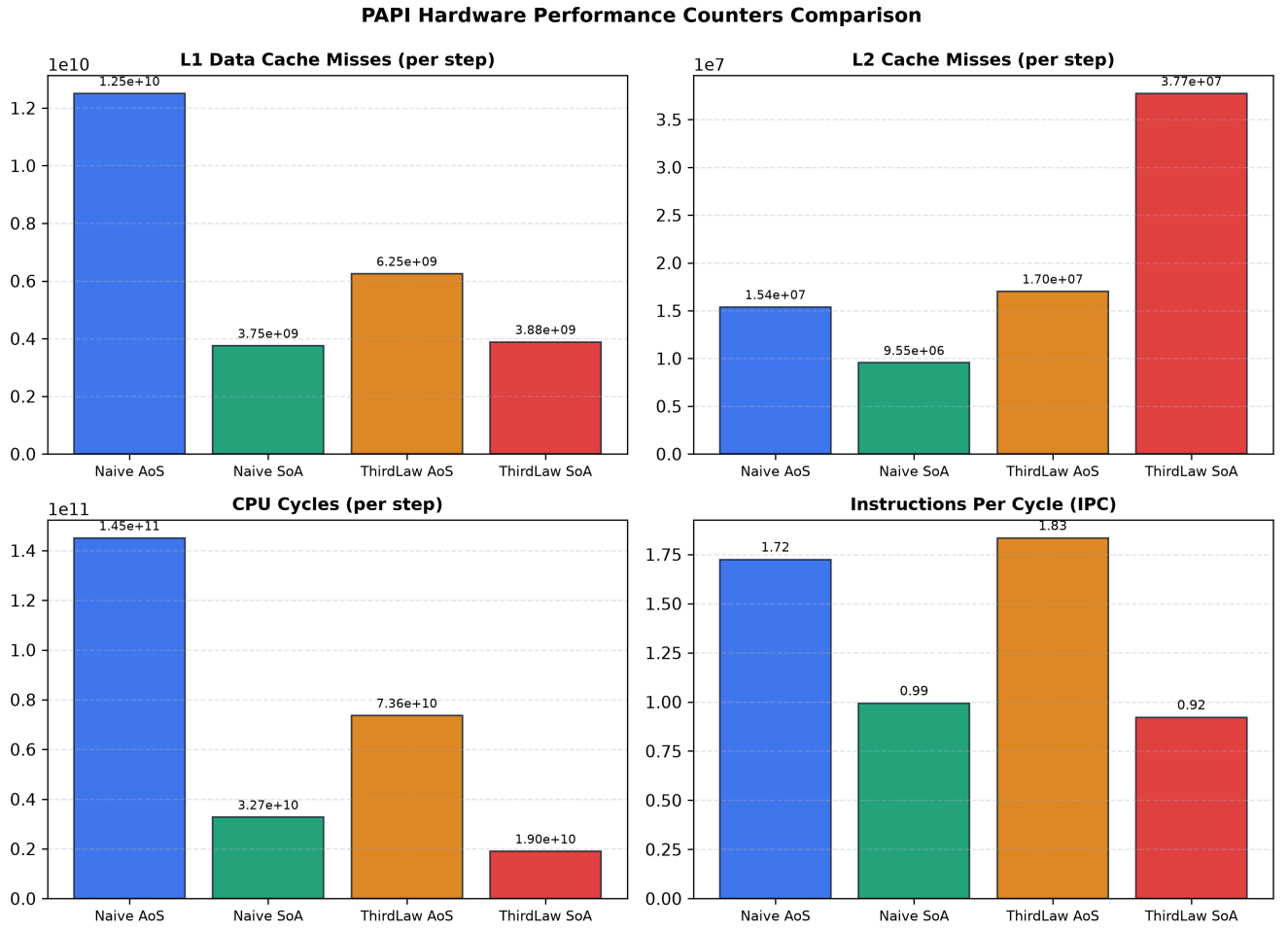


Figure 7: PAPI profiling results for the AoS and SoA kernels

Kernel	Speedup $\pm \sigma$	Maximum Relative Drift
Baseline	1.00x $\pm$ 0.011	4.35e-08
Naive	1.01x $\pm$ 0.0008	4.35e-08
ThirdLaw	1.80x $\pm$ 0.0427	4.35e-08
Rsqrt	1.89x $\pm$ 0.0021	4.74e-06
Blocks	1.01x $\pm$ 0.0014	4.35e-08
Rsqrt+Third	2.37x $\pm$ 0.2490	4.74e-06
Blocks+Third	1.85x $\pm$ 0.0053	4.35e-08
Blocks+Rsqrt	1.86x $\pm$ 0.0018	4.74e-06
Blocks+Rsqrt+Third	2.77x $\pm$ 0.0554	4.74e-06

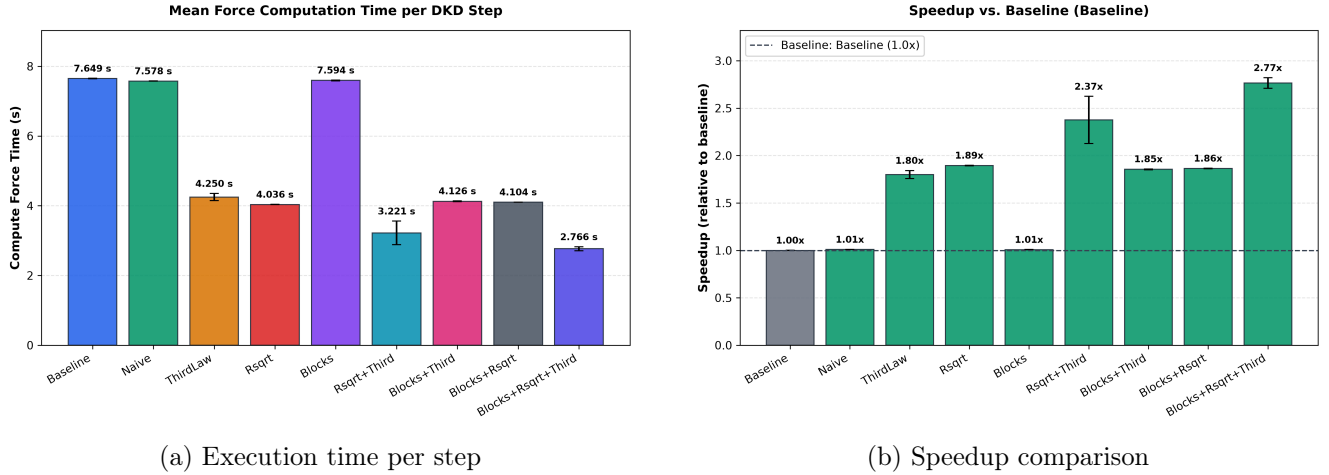


Figure 8: Performance comparison of the kernels in the Kernel experiment

4 OpenMP and MPI Strategies

In this section we will present and discuss the implementation of the DKD leapfrog algorithm using OpenMP (OMP) and MPI.

Both the approaches rely on the best-performing kernel discussed in Sec. 3.2.

Because setting up PAPI profiler for dealing with multi-thread and shared memory was very time-consuming, we have just relied on wall-time measurements and speedups for checking the well-behaviour of the kernels.

We will describe at first the kernels written using OMP in Sec. 4.1, we than will discuss the usage of MPI and the hybrid kernels MPI + OMP in Sec. 4.3.

4.1 OpenMP Kernels

For the OMP part, we considered the following three kernels: Blocks+Rsqrt (BR), Rsqrt+Third Law (RT) and Blocks+Rsqrt+Third Law (BRT).

We have chosen them because they perform well and are pretty easy to parallelize. We present the main aspects of each of them in the following paragraphs.

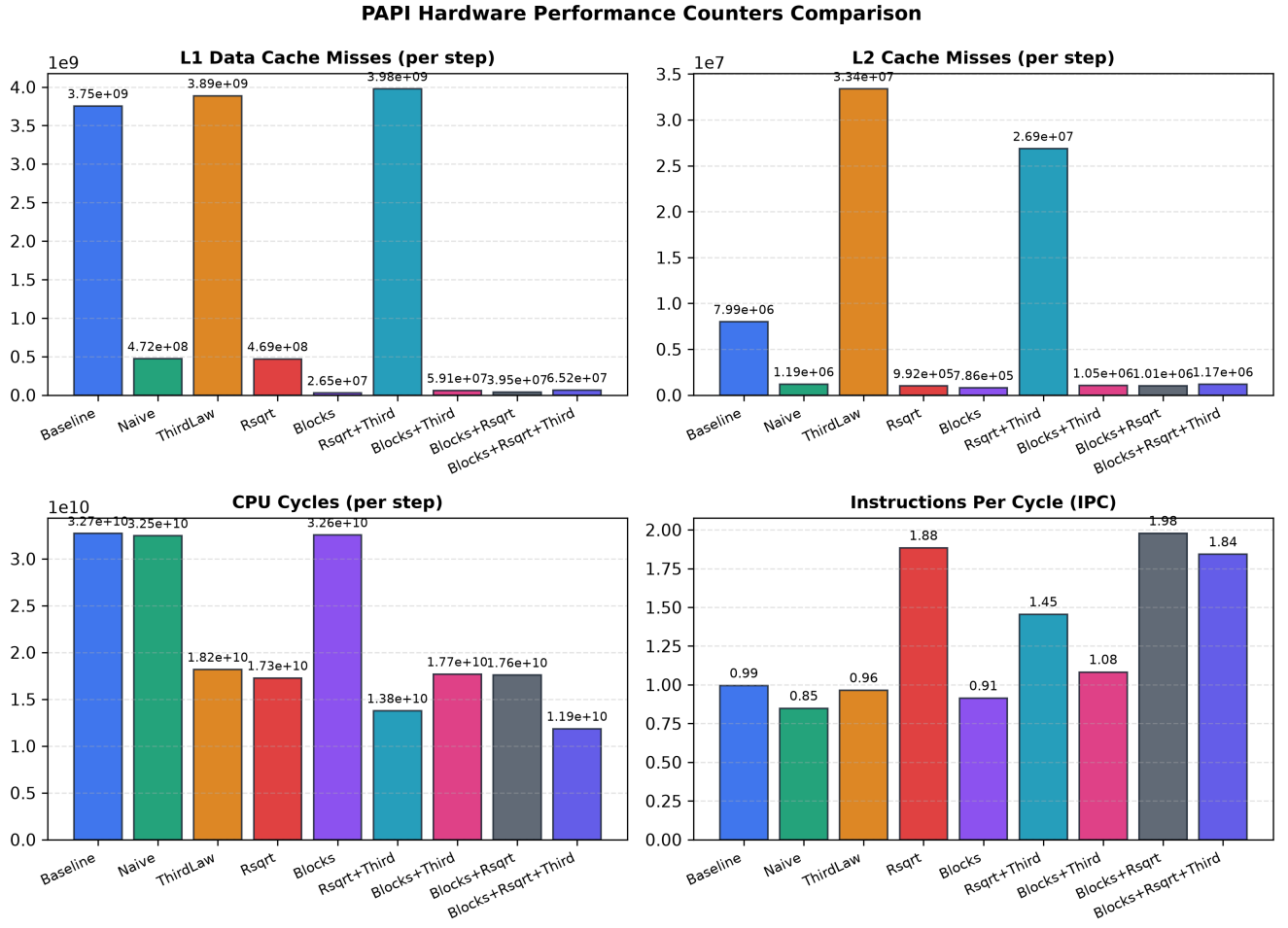


Figure 9: PAPI profiling results for the Kernel experiment

BR Kernel The main modification with respect to the serial version is the `# pragma omp parallel for` directive on the outermost loop.

We have also made explicit the `schedule(static)` directive: since each block does the same amount of job, it is fair to assign the same amount of chunks to each thread.

The kernel (as the others), parallelize also the initialization of the accelerations to 0.

RT Kernel This kernel differs a bit more from the serial one. Instead of the initialization of the acceleration arrays using `memset` as in the serial code we defined a private buffer for each thread using `calloc`, so that each thread can work on its data, while begin NUMA-friendly and avoiding false sharing.

At the end of the computation we run a global reduction phase to sum up the results.

Since that using this approach the innermost loop runs without data dependencies, the compiler is free to vectorize it: we have thus safely suggested the compiler to do so by using the directive `# pragma GCC ivdep`.

Since the Third Law exposes here a clear unbalance in the work done by each thread, we scheduled the parallel for using `schedule(dynamic, BLOCK_SIZE)`.

We have also implemented a “reduction” version in this kernel (“RTRed”) that substituted the thread buffers with an automatic array reduction suggested by the directive

```
reduction(+:ax[0:n],ay[0:n],az[0:n])
```

BRT Kernel This kernel simply merges the serial BRT kernel with the directives of the parallel RT. The main difference is that the dynamic scheduler chunk is set to 1. Also in this case we built a “BRTRed” kernel.

4.2 OpenMP Performances

We tested the OMP kernels for 100 steps and with 10^5 particles; we used here all the threads available in the Genoa node, i.e., 64.

Unsurprisingly, the computation time per step dropped from 10^0 s in the serial code to $\propto 10^{-2}$ in the parallel code. For an in-depth scalability analysis, refer to Sec. 6.

4.3 MPI Description

The MPI implementation required an approach which is totally different from both the serial and the OMP ones.

Indeed, we have not actually touched any of the acceleration kernels; instead, we modified both the main code and the `leapfrog_dkd_step` method to effectively work using MPI.

The main file initializes MPI, defines the group of threads with `MPI_Comm_rank(MPI_COMM_WORLD)` and their size; it also moves the initialized particles to having index 0 using `memmove` and finally runs the DKD algorithm. The specific implementation of the leapfrog DKD step exploits asynchronous communication by using `MPI_Isend` and `MPI_Irecv`, that are non-blocking instructions; we also allocate two buffers, the *buckets* in jargon: one of them holds the particles interested by the acceleration computation by the CPU, while the second is an empty space where the Network interface Card downloads the *next* batch of particles to use. In this way, we remove at most the cost of waiting for the information, and the

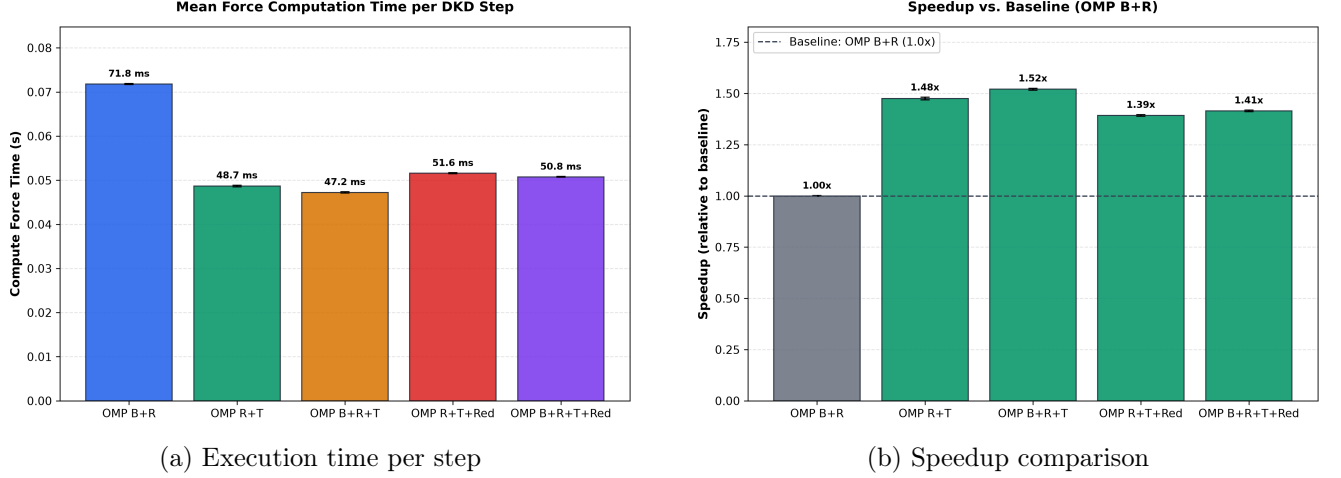


Figure 10: Performance comparison of the OpenMP kernels

total time is just the maximum between the computation time (heavier) and downloading time (lighter): the communication overhead is practically null.

We used Blocks+Rsqr as the only accelerations-kernel, since in this case defining the behaviour in case of having the Third Law would have been particularly delicate and maybe not effective too. Running the code on several MPI ranks reduced the total computation time per step of high factors, as we will see in Sec. 6.2

5 Containerization

In modern High-Performance Computing, containerization guarantees reproducibility and portability across different hardware architectures and operating systems: in order to achieve this, we bundled our N-Body simulation environment into a Linux-based container. We discuss here at first the definition of the Docker image in Sec. 5.1, and then discuss its deployment via Singularity/Apptainer in Sec. 5.2

5.1 Dockerfile

The environment was originally defined via a `Dockerfile` based on an `ubuntu:22.04` base image. This choice of a generic Ubuntu image, instead of a vendor-specific HPC image, guarantees maximum portability and avoids being stucked in proprietary libraries for a purely CPU-bound application.

We explicitly provisioned the container with the OpenMPI development libraries (`libopenmpi-dev`). Installing MPI *inside* the container is necessary since the compiler needs the MPI headers and linked-libraries to build the main binary. This internal MPI installation is completely bypassed at runtime.

When compiling the code inside the container, we adapted our compilation strategy. Specifically, we removed the `-march=native` flag from the `Makefile` and substituted it with `-march=x86-64-v3`. While this prevents the compiler from targeting the ultra-specific AVX-512 instruction set of the Orfeo Genoa nodes (i.e., the ones in `v4`), the `v3` microarchitecture level guarantees that vector extensions (AVX/AVX2) are still exploited.

This deliberate performance sacrifice ensures the resulting binary can run safely on any modern `x86_64`

machine without instruction faults.

Finally, the Docker images is passed to GitHub Container Registry and it is available inside the code repository for the project.

5.2 Apptainer

Docker requires root privileges to mount volumes and manage namespaces, thus is not natively supported on Orfeo. To overcome this, we adopted **Apptainer**.

We pulled and converted our Docker image into a Singularity Image Format (.sif) file directly on the Orfeo login nodes.

The containerized application is executed via SLURM batch scripts using the `singularity exec` command. For MPI execution, we wrap the host's MPI launcher around the containerized binary:

```
singularity exec nbody.sif mpirun -np $SLURM_NTASKS ... /app/src/main
```

This step is very critical, since Singularity must bind the host's MPI libraries over the container's internal MPI at runtime.

If the container were to use its internal OpenMPI instance, it would lack the drivers required to interface with Orfeo's InfiniBand RDMA network, causing the ring-shift communications to silently fall.

Instead, by injecting the host MPI, the containerized binary runs without any relevant loss of performances.

6 Scalability Analysis

In this section we present the scalability analysis for OMP (6.1), MPI (6.2), vectorization (6.3) and the effects of containerization on performances (6.4).

The analysis on OMP and MPI is ran in native, non-containerized, code, in order to not saturate performances with the container's launch overhead.

We tested the OMP and MPI code on a single Genoa node but using, respectively, increasing numbers of threads and MPI ranks.

We reasoned here on both strong and weak scalability.

In the former the problem size is kept fixed to $N = 10^5$ particles. For the *Amdahl's Law*, the maximum speedup $\mathcal{S}$ achievable is:

$$\mathcal{S}(n, N) = \frac{t(1, N)}{t(n, N)} \rightarrow \frac{1}{(1 - f) + \frac{f}{n}} \xrightarrow{n \rightarrow \infty} \frac{1}{1 - f} \quad (1)$$

where: n is the total number of resources, $t(k, N)$ is the time of running the code having size N on k resources and f is the serial fraction of the algorithm.

We also compute the efficiency $\mathcal{E}(n, N) = \mathcal{S}(n, N)/n$.

Weak scalability measures the effects of keeping constant the workload assigned to each resource. Since the serial DKD algorithm scales with $\mathcal{O}(N^2)$, the workload $\mathcal{W}$ per each resource reads as:

$$\mathcal{W} = \frac{N_n^2}{n} \quad (2)$$

So to keep $\mathcal{W}$ fixed, at the increase of n , we multiplied N by a factor $\sqrt{n}$.

We figured out that performances measured by starting with $N_1 = 10^4$ as suggested by the exercise were too much noisy, thus also in this case we set $N_1 = 10^5$, and we scale it up to $N_{64} = 8 \times 10^5$.

In this scenario, we plot the weak efficiency, defined as $\mathcal{E}_w(n) = \frac{t(1, N_1)}{t(n, N_n)}$.

Looking at the code implementation, the main reasons for possible slowdowns due to the computation of the energy drift check (in particular, the computation of the kinetic energy, which runs linearly in $\mathcal{O}(N)$), which is the only component of the algorithm which has not been parallelized.

The other components of the code, as the plot will suggest, are not relevant for the total running time.

6.1 OpenMP Scaling

We have ran scaling analysis on the OMP BRT kernel, with threads in the set $\{1, 2, 4, 8, 16, 32, 64\}$, i.e., up to the maximum number of threads provided by the Genoa node.

Strong scalability show an almost complete adherence to the target maximum speedup, with a maximum drift of 12.5% only when working with 64 threads. This high value is symptom of a very well-working kernel.

The tiny slow down is mainly due to the computation of the energy drift, whose impact grows at the increase of N , as Fig. 12 suggests, from the 8% having just on thread, to 17% having 16 threads, stabilizing then on those figures. In the tests, drift is only computed at the beginning (and serves here as a “target”) and at the final step of the algorithm. Repeating the check at each iteration or “once in a while” constitutes a severe bottleneck on the performances.

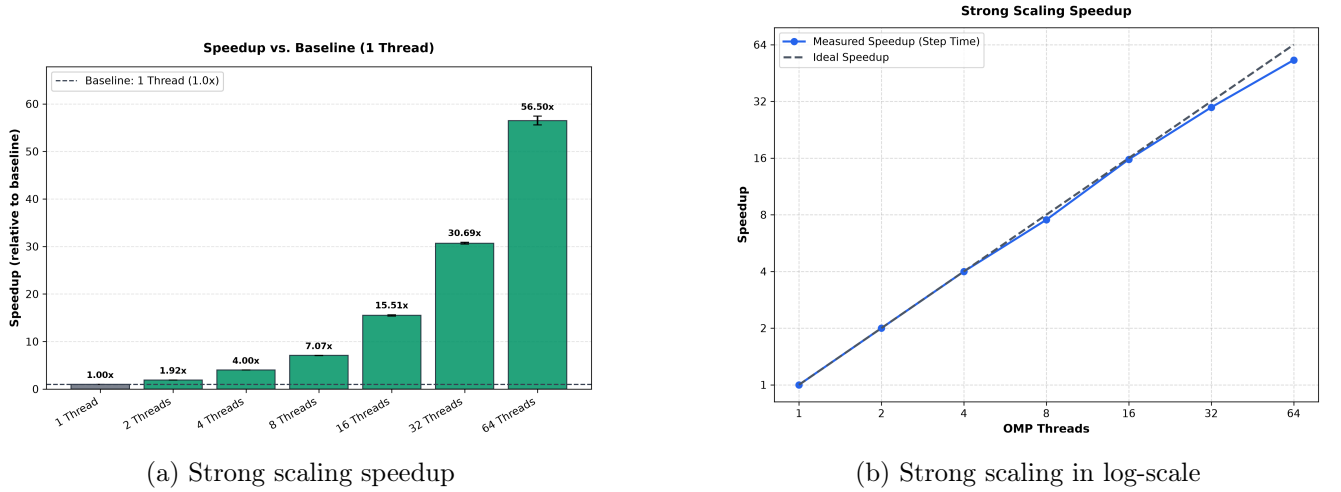


Figure 11: Strong scalability of the OpenMP BRT kernel

Regarding weak scalability, we observed that the execution time per step remains nearly constant, but surprisingly yields speedups (slightly) exceeding 1.0x (“hyper-performance”).

We can attribute this behavior to both communication anomalies and to some particular cache-efficiency effect.

In our weak scaling model the total memory footprint of the simulation grows sub-linearly, at a rate of $\mathcal{O}(\sqrt{P})$.

Conversely, as we distribute the workload across P threads, we harness P physical cores, meaning the

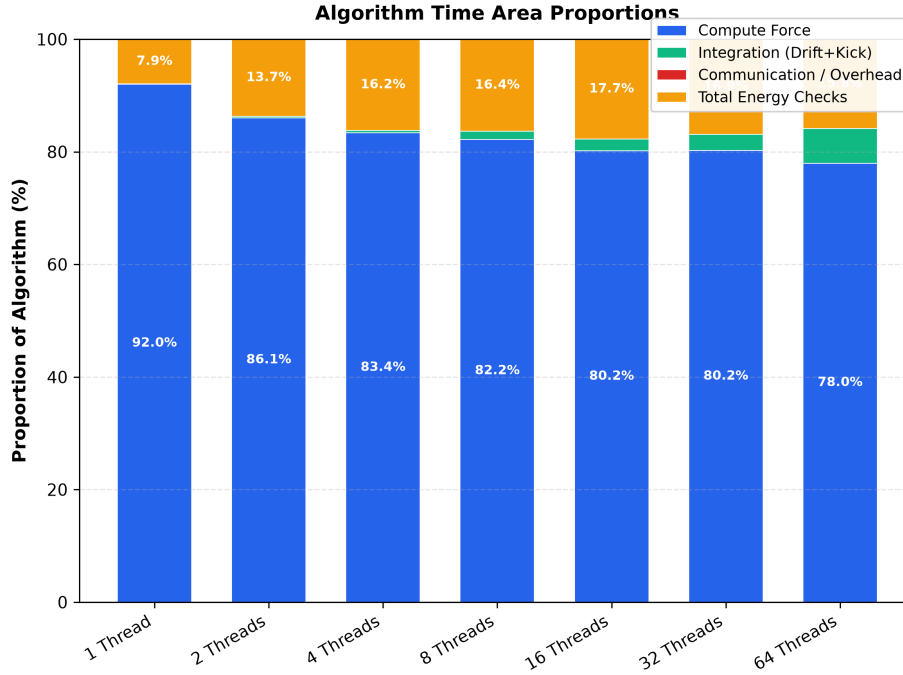


Figure 12: Proportion of time spent doing different activities during strong scalability analysis

total aggregate L2 and L3 cache capacity available to the program grows linearly, at a rate of $\mathcal{O}(P)$. Because the aggregate cache capacity outpaces the memory footprint ($\mathcal{O}(P) > \mathcal{O}(\sqrt{P})$), scaling up to 4-16 threads allows a significantly larger fraction of the global working set to reside permanently in L2 and L3 caches, drastically reducing DRAM bandwidth bottlenecks, accelerating the memory-bound memory-accesses and thus providing an efficiency greater 100%.

At higher thread counts (e.g., 64 threads), the inevitable OpenMP synchronization overhead and NUMA boundary crossings finally offset this cache advantage, normalizing the efficiency back towards the ideal curve.

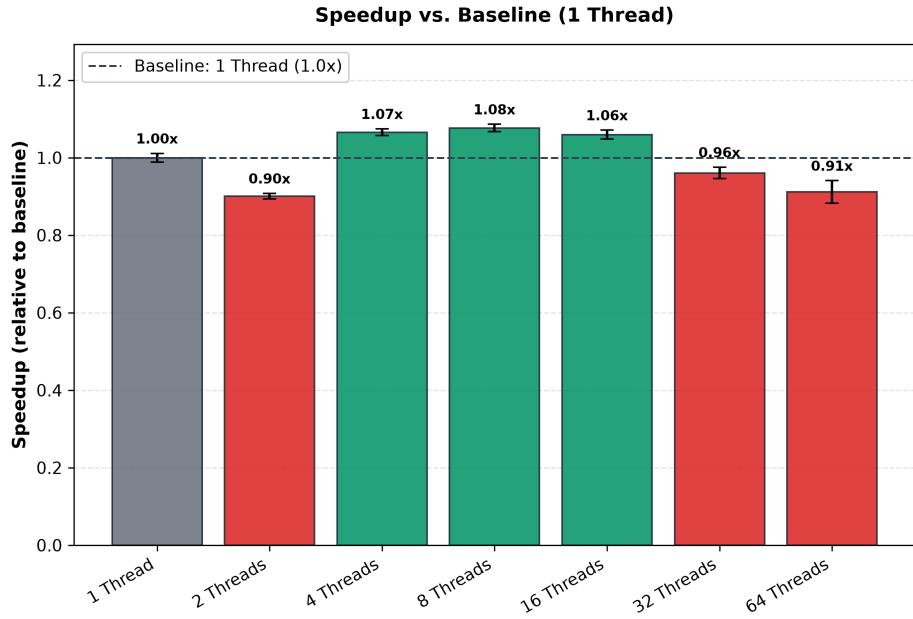
6.2 MPI Scaling

To test strong and weak scalability of the MPI kernels we used MPI ranks in the set $\{1, 2, 4, 8, 16, 32, 64, 128\}$.

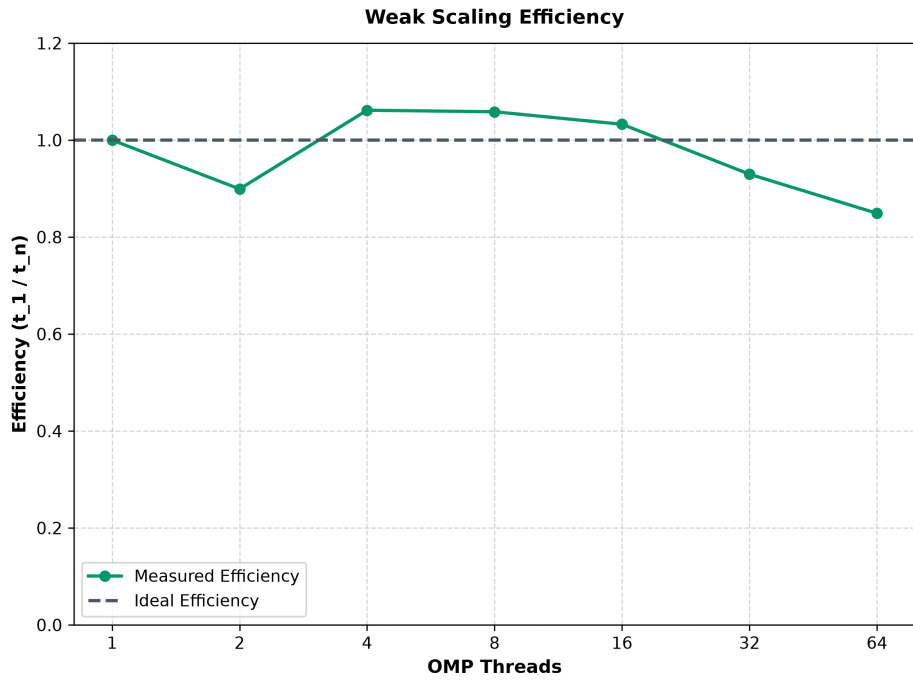
If we consider per-step measurements, both the analysis show extremely good performances: regarding strong scalability, we observed a speedup of 116x in the case of 128 ranks, meaning an efficiency $\approx 90\%$ (Fig. 14).

For weak scalability, we observed that the efficiency decreases very slowly, being still higher than 90% on 128 MPI ranks (Fig. 15).

Instead, if we consider the total time, performances decrease dramatically when energy drift checks are computed: as one can see from both the panels in Fig. 16 the performances degradates up to 20%: once again computing the check slows down the code and completely destroys the capability.

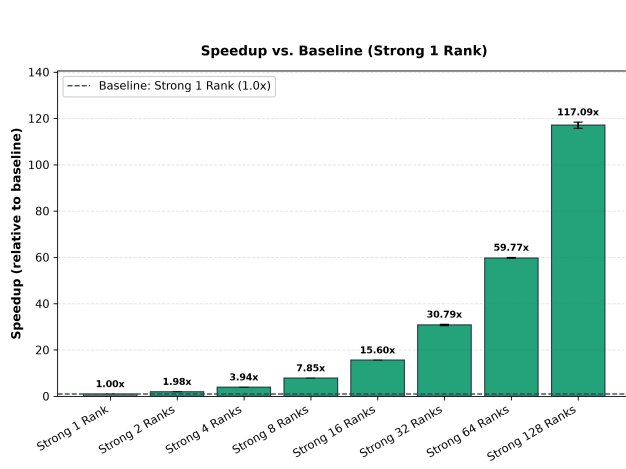


(a) Weak scaling speedup

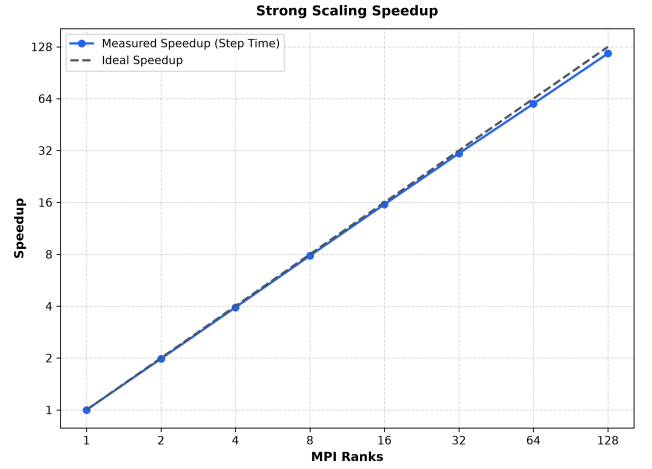


(b) Weak scaling efficiency in log-scale

Figure 13: Weak scalability of the OpenMP BRT kernel

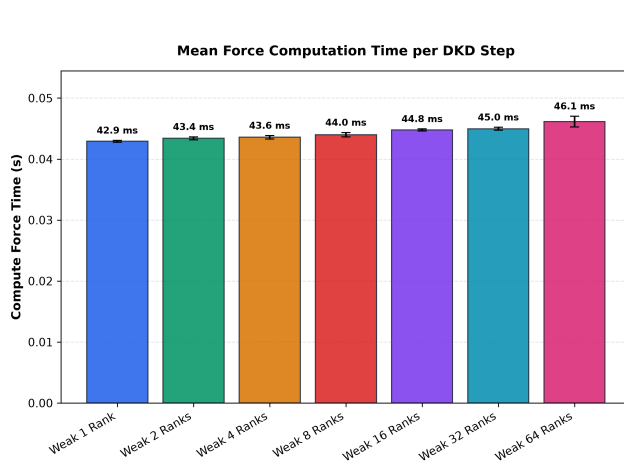


(a) Strong scaling speedup

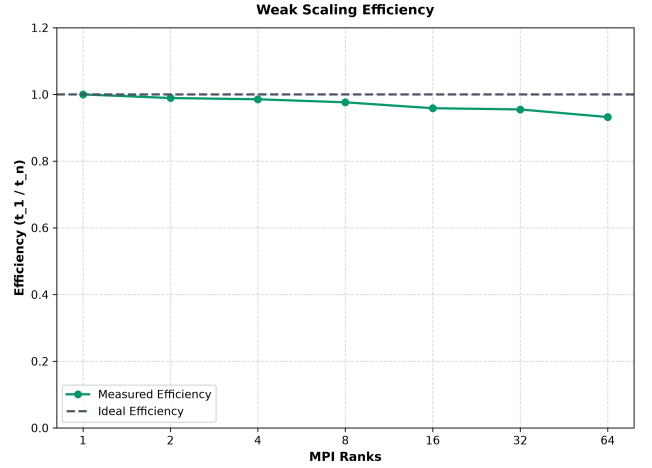


(b) Strong scaling in log-scale

Figure 14: Strong scalability of the MPI implementation

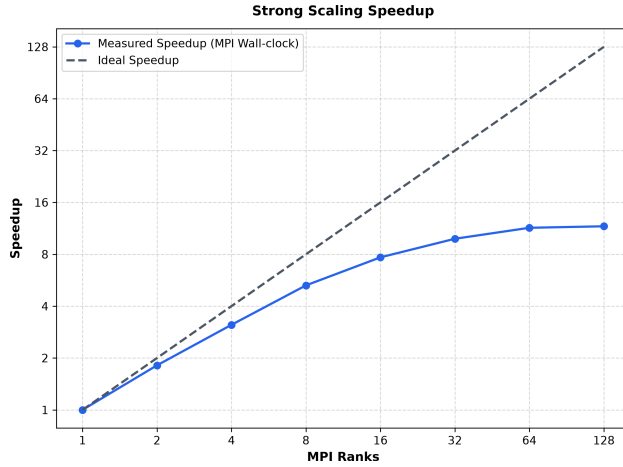


(a) Weak scaling execution time per step

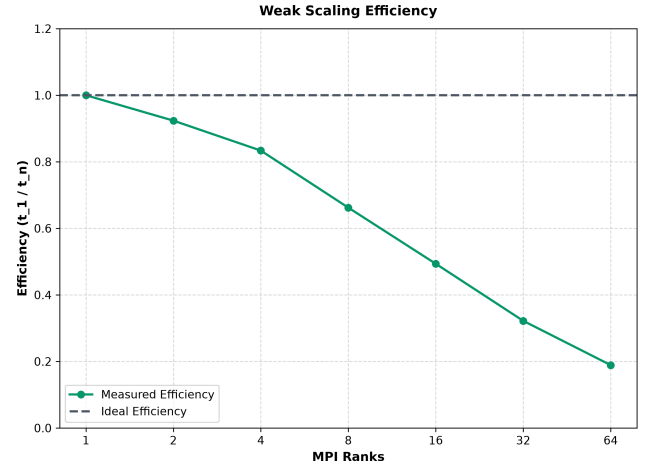


(b) Weak scaling efficiency in log-scale

Figure 15: Weak scalability of the MPI implementation



(a) Strong scaling with energy drift checks



(b) Weak scaling with energy drift checks

Figure 16: Impact of energy drift checks on MPI scaling efficiency

6.3 Vectorization

In both the scaling analysis for OpenMP and MPI approaches we would say that one of the main reasons why the code keeps running quite fast and scales accordingly is related to the use of the `-ffast-math` flag that enables vectorization and ensures alone a speedup of around 2.5x with respect to the code without it.

Notably, *without* the flag a quick strong scalability analysis shows an efficiency of more than 97% ($S = 125x$) when working with 128 MPI ranks (Fig. 17).

We ran the MPI kernel using `-fopt-info-vec-all=report.txt`; the report revealed that vectorization happens only when suggested through the pragma `GCC ivdep` in the innermost loop of the OMP+BR kernel used in MPI and in the kernel used when the size of the MPI rank is 1 (always OMP+BR kernel, without `GCC ivdep` pragma); moreover, some code in `generate_initial_condition.c`, in `particles.c` and in the computation of the trimmed mean in `profiling.c`. As already said, this minimal vectorization still has a strong impact on the performances of the code.

6.4 Container Scaling

The exercise required to compute the overhead due to the use of containerization. We ran the following three configurations: *Native AVX-512*, having the compilation flag `-march=native`, *Native AVX-2*, with the compilation flag `-march=x86-64-v3` and finally the *Container AVX-2*.

The report below, ran on 1 MPI rank and 1 thread, with $N = 10^5$ particles, for 25 steps, shows the per-step mean computation time:

Target	Step Time (ms) $\pm \sigma$	Speedup
Native AVX-52	5193.31 $\pm$ 1.67	1.00x
Native AVX-2 (v3)	5206.43 $\pm$ 8.17	0.99x
Container AVX-2 (v3)	5278.86 $\pm$ 4.94	0.98x

As one can see, the gap is minimal; in the first case is mainly due to ISA gap because of the use of a

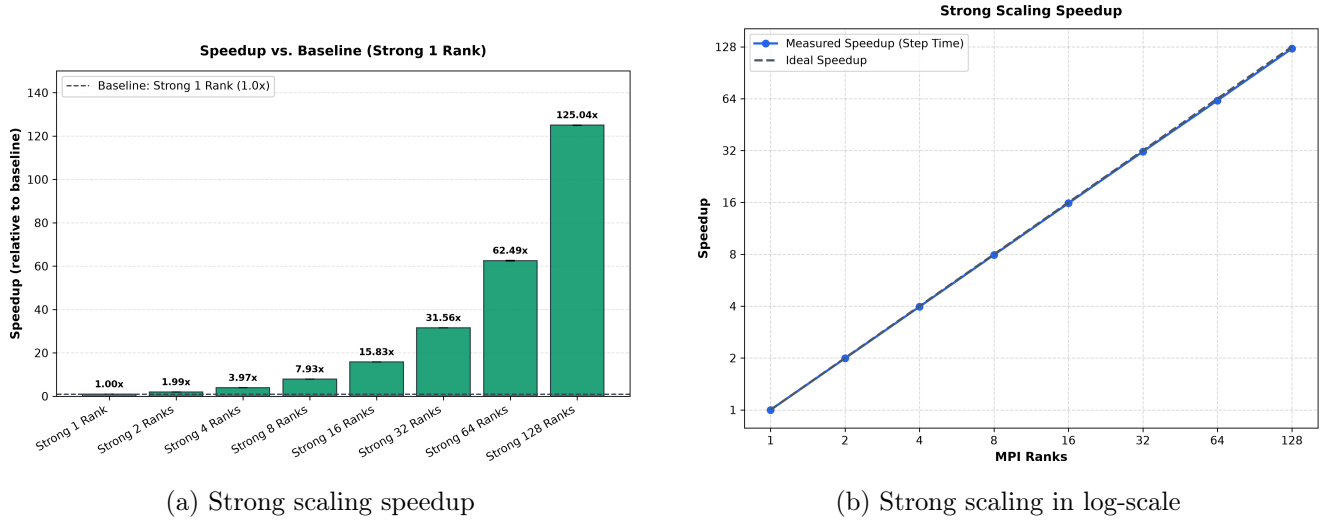


Figure 17: Impact of the `-ffast-math` flag on MPI strong scaling

AVX-2 target; the second gap is instead pure containerization overhead.

Regarding scalability, containerization show a loss of performances per step consistently around 20% with respect to each corresponding native code running on the same number of MPI ranks. Something similar happens when considering weak scalability, where the loss is always around 20%. One main reason for this slowdown is due to the fact that during containerized execution, OpenMPI reported that the Cross-Memory Attach (CMA) single-copy mechanism was disabled due to Linux namespace isolation (`btl_vader_single_copy_mechanism CMA is not available due to different user namespaces`). Consequently, intra-node communication fell back to a slower shared-memory protocol, exposing the library and IPC binding overhead introduced by Apptainer.

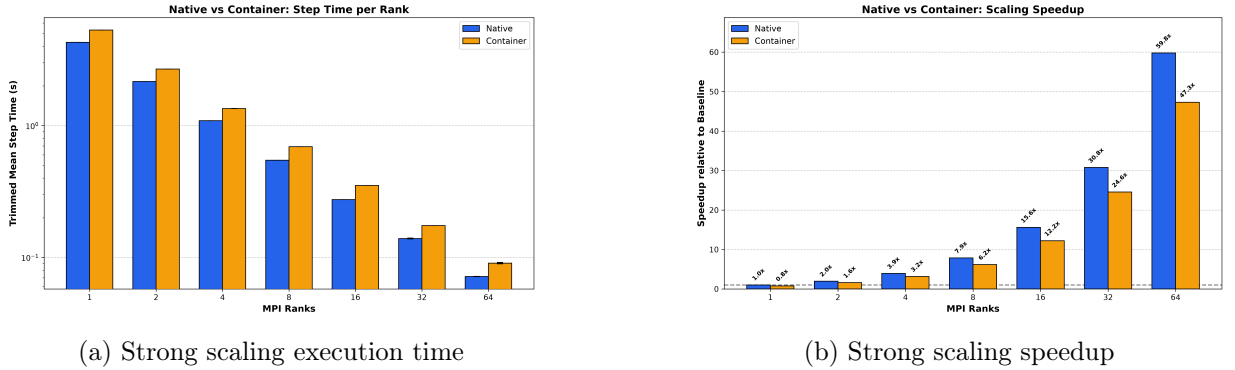
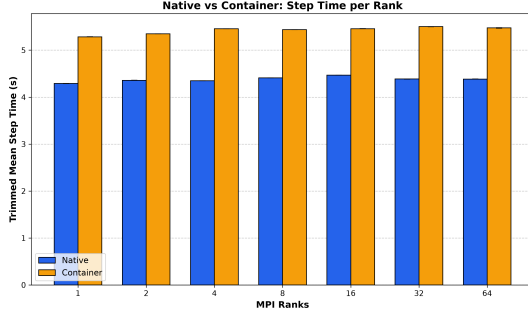


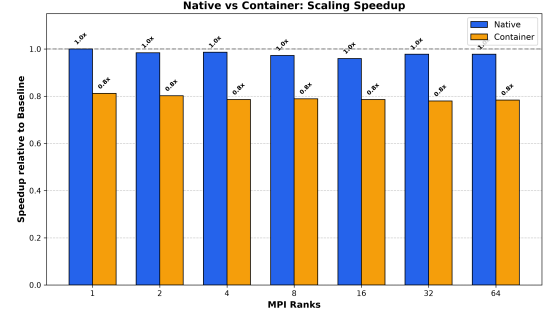
Figure 18: Execution time and speedup for container strong scaling

7 Conclusion

In conclusion, for this exercise we have parallelized a Direct N-Body gravitational simulation problem using both shared- and distributed- memory approaches.

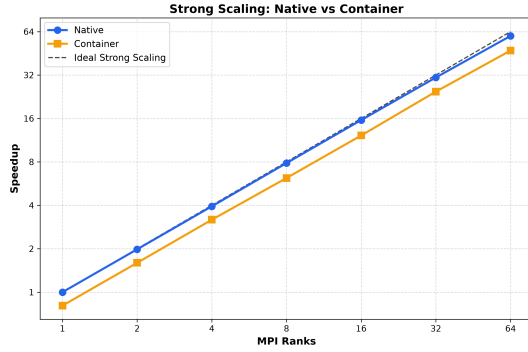


(a) Weak scaling execution time

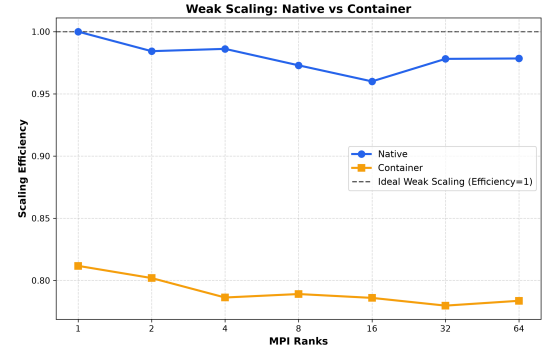


(b) Weak scaling speedup

Figure 19: Execution time and speedup for container weak scaling



(a) Strong scaling



(b) Weak scaling

Figure 20: Strong and weak scaling of the containerized application

We at first improved the single-core engine code, identifying bottlenecks and criticalities; we then selected the best-performing kernels and parallelized them using OpenMP and MPI libraries.

We have then conducted extensive analysis on the scalability and performances of the different approaches, including also a brief discussion on vectorization.

We finally realized a container for portability and reproducibility of the code using Docker and App-tainer/Singularity, before concluding with a quick analysis of their induced overhead.

Brief clarifications In the report I have always speak using the first plural for “we”, “us”, etc. This has to be intended as just a choice of style.

Regarding the use of Generative AI tools in this project, I mainly used Claude Opus 4.6 and Gemini Pro 3.1 to help me debugging the code. They have also provided the code for the plots in this report, as well as for optimizing the FMA kernel, as already stated in its discussion (Sec. 3.1).